Automated Source Instrumentation & Path Profiler

Major Assignment: Runtime Call Flow and Execution Frequency Extraction

Student: Omar Ahmed Abdel Hameed (ID: 24P0201) • Instructor: Dr. Islam El-Maddah

1. Overview & Architectural Objectives

Software performance evaluation and tracing require precise observation of control flow across subroutines. This project implements an instrumentation-based execution profiler for procedural C/C++ architectures. Rather than relying on invasive manual strings or external binary rewriting, the design employs the Resource Acquisition Is Initialization (RAII) idiom combined with standard macro reflection (`__func__`) to track dynamic function calls.

The profiling harness satisfies two operational deliverables:

- **Chronological Event Trace:** Recording real-time transitions (`ENTER` and `EXIT`) with explicit caller-callee bindings.
- **Path Profile Aggregation:** Maintaining runtime call-edge counters in an internal hash table to quantify call frequencies between subroutines upon program termination.

2. Instrumentation Design & Engine Mechanics

The system is divided into two core components: a centralized tracking engine (`Profiler` singleton) and a lightweight scope guard (`ScopeProfiler`).

```
      // RAII Scope Guard Mechanism struct ScopeProfiler { const char* func_name;
explicit ScopeProfiler(const char* name) : func_name(name) {
Profiler::instance().enter(func_name); } ~ScopeProfiler() {
Profiler::instance().exit(func_name); } }; #define PROFILE_FUNCTION()
ScopeProfiler _prof_scope_guard(__func__)
```

Placing `PROFILE_FUNCTION();` at the opening of any subroutine automatically pushes the symbol name onto a dynamic call stack during construction. When the subroutine finishes execution—regardless of control branches or early returns—the destructor fires, emitting an `EXIT` log and safely popping the stack frame.

3. Test Target Program Structure

A multi-tier procedural program was constructed to validate call paths and nested dependencies without recursive loops:

Subroutine	Direct Invocations	Target Role & Operations
`main()`	`parse_input()`, `compute_results()`	Root orchestration driving the sequential pipeline.
`parse_input()`	`step_a()`	Initial preprocessing phase.
`step_a()`	`step_b()`	Intermediate transformation layer.

Subroutine	Direct Invocations	Target Role & Operations
`compute_results()`	`step_b()`	Analytical computation phase sharing common utility routines.
`step_b()`	*Leaf Function*	Shared execution target invoked across multiple distinct branches.

4. Execution & Output Verification

The instrumented application was compiled and executed under CLion using the native toolchain. The terminal output demonstrates correct, orderly execution of all workflow steps.



Figure 1: Standard console output confirming sequential workflow execution without runtime anomalies.

5. Generated Event Log Trace (Sequential Events)

The instrumentation system stream-writes all entry and exit occurrences to `event_log.txt`. Each function entry records its exact caller by querying the top of the internal call stack.



Figure 2: Inspection of event_log.txt displaying complete caller-callee bindings across call and return steps.

6. Generated Path Profile (Edge Frequency Aggregation)

During teardown, the profiler iterates over all registered caller-to-callee edges and summarizes their execution frequencies in `path_profile.txt`.



Figure 3: Inspection of path_profile.txt confirming correct aggregation of path transitions and call counts.

7. Summary & Compliance

The implementation provides automated instrumentation, zero manual string entry, and full tracking of call paths and events. All source files (`profiler.hpp`, `main.cpp`, `CMakeLists.txt`), output text files, and this report are compiled in the final archive.